VueJS

From zero to a real SPA — in one day.

ITI · AHMED GHORABA · 7–9 HRS

ABOUT ME

Ahmed Ghoraba

Software engineer · ITI teaching assistant

- I write Code for a living — not in theory, in production.
- I'll be coding alongside you. Interrupt whenever.
- Teams / WhatsApp / raise your hand — whatever's fastest.

THE DAY

What we'll cover

01 What Vue is, why it exists

02 Setup: Vite, DevTools, daisyUI

03 SFCs & app structure

04 Template syntax & directives

05 Reactivity — deep

06 Components, props, emits

07 Slots

08 Lifecycle hooks

09 Vue Router

10 HTTP & APIs

11 Composables

12 Pinia — state management

13 Forms

14 TypeScript, env, deploy

15 The map — what's next

🍕 Lots of demos & breaks

END OF THE COURSE

You'll have shipped a real SPA

Real components

Props, emits, slots. Not a todo list.

Real routing

Multi-page SPA with nested routes and guards.

Real state

A cart that survives reload — like actual e-commerce.

BEFORE WE START

Ground rules

- **Interrupt.** If something isn't clicking, we stop.
- **Don't skip the demos.** That's where it clicks.
- **Google is allowed.** Reading docs is a skill.
- **Side talk is forbidden.** Respect the room.
- **AI-written lab code = 0.** Non-negotiable.

PART 01 / 15

Why Vue?

What it is, and why you'd pick it.

WHAT IT IS

Vue is a progressive JavaScript framework

Built by **Evan You**, a former Google engineer who wanted to extract the best parts of AngularJS into something lighter and easier to reason about.

Today, Vue 3 powers apps at GitLab, Adobe, Xiaomi, Trivago, and thousands more.

"PROGRESSIVE" MEANS

Start small. Scale when ready.

Sprinkle it in

Drop Vue onto one existing page. No build step needed.

Build an SPA

Vite + Router + Pinia. The default for most apps.

Full stack with Nuxt

SSR, file-based routing, serverless. When you need SEO.

SPA

One page. Many screens. No reload.

A **Single Page Application** loads once, then JavaScript swaps the content as the user navigates.

`Gmail` `Google Docs` `Netflix` `X (Twitter)` `Notion` `Figma`

SPA FIT CHECK

Use an SPA when…

✓ Good fit

- Dashboards & admin tools
- Social apps, editors, chats
- Anything with lots of interaction
- Complex filters & state

✗ Bad fit

- Marketing landing pages (use static)
- Blogs & content-heavy sites (use SSR)
- SEO-critical pages without SSR
- JavaScript-averse audiences

IN PRODUCTION

You're learning a real tool

GitLab	Adobe	Xiaomi	Trivago
Nintendo	Apple Music (web)	BMW	Louis Vuitton

PART 02 / 15

Setup

From nothing to a running Vue app in 3 minutes.

WHAT YOU NEED INSTALLED

The toolchain

Node + npm

JS outside the browser. Needed for everything.

`node -v` → v20+

Vite

Dev server + build tool. Fast. Hot reload. Modern.

Vue DevTools

Ships with `npm create vue@latest`. Auto-installed.

ONE COMMAND

Scaffold a Vue project

terminal

```
# one-liner to scaffold + install
npm create vue@latest

# follow the prompts:
✓ Project name:            my-shop
✓ TypeScript?               No      // we'll skip today
✓ JSX?                      No
✓ Vue Router?               Yes     // ← yes!
✓ Pinia?                    Yes     // ← yes!
✓ Vitest / E2E / ESLint?    No      // add later

cd my-shop && npm install && npm run dev
```

TOUR

What's in the box

```
my-shop/
├── public/
├── src/
│   ├── assets/
│   ├── components/
│   ├── composables/   // ← you'll add
│   ├── router/
│   ├── stores/
│   ├── views/
│   ├── App.vue
│   └── main.js
├── index.html
├── package.json
└── vite.config.js
```

- `src/` — **where you live**. Everything else is plumbing.
- `components/` — reusable pieces.
- `views/` — page-level components that routes map to.
- `public/` — files served as-is (favicon, robots.txt).

INSTALL IT. NOW. SERIOUSLY.

Vue DevTools is non-negotiable

- See the whole component tree, live.
- Inspect props, refs, computed — everything reactive.
- See router state & route params.
- See Pinia stores & mutations.
- Time-travel through state changes.

OPTIONAL BUT RECOMMENDED

daisyUI + Tailwind

Pre-styled components. We don't want to burn today styling buttons.

terminal

```
npm install tailwindcss @tailwindcss/vite daisyui
```

vite.config.js

```
import { defineConfig } from 'vite'
import vue from '@vitejs/plugin-vue'
import tailwindcss from '@tailwindcss/vite'

export default defineConfig({
  plugins: [vue(), tailwindcss()]
})
```

DEMO 01

Hello, Vue.

Scaffold the app, run it, see the welcome screen.
10 minutes, live.

PART 03 / 15

SFC & Structure

The three-file mental model.

SINGLE FILE COMPONENT

One file. Three blocks.

Hello.vue

```vue
<template>
  <h1>Hello, {{ name }}</h1>
</template>

<script setup>
import { ref } from 'vue'
const name = ref('Ahmed')
</script>

<style scoped>
h1 { color: #42b883; }
</style>
```

THE SHELL

index.html

index.html

```
<!doctype html>
<html>
  <head>
    <title>My Shop</title>
  </head>
  <body>
    <div id="app"></div>   // ← Vue mounts here
    <script type="module"
            src="/src/main.js"></script>
  </body>
</html>
```

- Just **one real thing** matters: `<div id="app">`
- That's where Vue renders everything.
- Could you drop jQuery in here? Yes. **Don't.**

THE BOOT FILE

`main.js` — where it all starts

src/main.js

```
import { createApp } from 'vue'
import { createPinia } from 'pinia'
import App from './App.vue'
import router from './router'

const app = createApp(App)    // create Vue instance from root component

app.use(createPinia())       // plug in Pinia (state)
app.use(router)               // plug in Router

app.mount('#app')            // render into <div id="app">
```

THE ROOT COMPONENT

App.vue

src/App.vue

```vue
<template>
  <Navbar />
  <main>
    <RouterView />          // routes render here
  </main>
  <Footer />
</template>

<script setup>
import Navbar from './components/Navbar.vue'
import Footer from './components/Footer.vue'
</script>
```

Keep it small — layout + RouterView. The real work happens in child components.

Composition API vs Options API

Options API (the old way)

```
export default {
  data() { return { count: 0 } },
  computed: {
    double() { return this.count * 2 }
  },
  methods: {
    inc() { this.count++ }
  }
}
```

Composition API (today)

```
<script setup>
import { ref, computed } from 'vue'

const count = ref(0)
const double = computed(() => count.value * 2)
const inc = () => count.value++
</script>
```

OUR CHOICE

We teach Composition API

- Scales better — logic groups by concern, not by option type.
- Composes better — extract to composables, reuse anywhere.
- Plays nicer with TypeScript.
- It's where the ecosystem is going. All official libs are written this way.

PART 04 / 15

Template Syntax

HTML with superpowers.

TEMPLATE

Mostly HTML. Plus directives.

Vue templates are **real HTML**. The browser parses them. On top of HTML, Vue adds:

- **Mustache interpolation** — `{{ value }}` — for text.
- **Directives** — attributes starting with `v-` — for behavior.

`{{ }}` — text interpolation

```
<h1>Hello {{ name }}</h1>
<p>You have {{ items.length }} items</p>
<p>Total: ${{ (price * qty).toFixed(2) }}</p>
```

> Anything inside `{{ }}` is a **JavaScript expression**. Not a statement — no if, no let.

`v-bind` — attribute binding

```
// long form
<img v-bind:src="product.image" v-bind:alt="product.name" />

// shorthand — you'll see this everywhere
<img :src="product.image" :alt="product.name" />

// dynamic class and disabled
<button :class="{ active: isOpen }" :disabled="loading">
  Buy
</button>
```

`v-on` — event listeners

```
// long form
<button v-on:click="handleClick">Click</button>

// shorthand — use this
<button @click="handleClick">Click</button>

// inline expression
<button @click="count++">+1</button>

// pass args
<button @click="buy(product.id)">Buy</button>
```

`v-model` — two-way binding

The gold. Bind an input to state. User types → state updates. State changes → input updates.

```
<input v-model="email" placeholder="you@iti.eg" />
<p>You typed: {{ email }}</p>

// modifiers
<input v-model.trim="name" />       // trim whitespace
<input v-model.number="age" />       // cast to number
<input v-model.lazy="bio" />         // sync on blur, not keystroke
```

`v-if` / `v-else` / `v-show`

```
<div v-if="user">Welcome, {{ user.name }}</div>
<div v-else-if="loading">Logging in…</div>
<div v-else>Please log in.</div>

// v-show toggles display:none — stays in DOM
<div v-show="isMenuOpen">…menu…</div>
```

> **Rule of thumb:** `v-if` when it's rarely true. `v-show` when it toggles often.

`v-for` — loops

```
<ul>
  <li v-for="product in products" :key="product.id">
    {{ product.name }} — ${{ product.price }}
  </li>
</ul>

// with index
<li v-for="(item, i) in items" :key="item.id">
  {{ i }}. {{ item.name }}
</li>
```

> **ALWAYS give `:key` a stable unique ID.** Not the index. Vue uses it to track what changed.

Modifiers — little tweaks

MODIFIER	WHAT IT DOES	EXAMPLE
`.prevent`	Calls `preventDefault()`	`@submit.prevent="save"`
`.stop`	Calls `stopPropagation()`	`@click.stop="…"`
`.once`	Fires only once	`@click.once="init"`
`.lazy`	v-model syncs on blur, not keystroke	`v-model.lazy`
`.trim`	Strips whitespace from v-model value	`v-model.trim`
`.number`	Casts v-model to number	`v-model.number`
`.enter`	Key modifier — only on Enter	`@keyup.enter="search"`

KEEP THIS OPEN

Your daily directives

DIRECTIVE	SHORTHAND	PURPOSE
`v-bind:attr`	`:attr`	Bind a JS value to an HTML attribute.
`v-on:event`	`@event`	Listen to a DOM event.
`v-model`	—	Two-way binding for form inputs.
`v-if / v-else`	—	Conditional rendering (adds/removes from DOM).
`v-show`	—	Toggles display:none.
`v-for`	—	Loop over arrays / objects. Always with `:key`.
`v-html`	—	Render raw HTML. Dangerous — XSS risk.
`v-text`	—	Same as mustache. Rarely used.

DEMO 02

Directives.

Counter + toggle + list + live input.
One component. Every directive.

PART 05 / 15

Reactivity

The part where Vue earns its keep.

THE PROBLEM

Plain variables don't tell anyone they changed

```
let count = 0
count++                                  // DOM has no idea anything happened

document.querySelector('#count').textContent = count    // manual, painful
```

Vue's whole job: make variables that **announce their changes**, so the UI can re-render itself.

`ref()` — the most common reactive primitive

```
import { ref } from 'vue'

const count = ref(0)           // wrap a value
const name  = ref('Ahmed')
const items = ref([])

// read / write in script: .value
count.value++
name.value = 'Ghoraba'
items.value.push({ id: 1 })
```

THE #1 BEGINNER BUG

The `.value` footgun

✗ In script

```
const count = ref(0)

count++              // ❌ forgot .value
count.value++        // ✓ yes
```

✓ In template

```
<p>{{ count }}</p>          // ✓ auto-unwrapped
<p>{{ count.value }}</p>    // ❌ wrong here
```

Script has .value. · Template doesn't.

`reactive()` — for objects

```
import { reactive } from 'vue'

const user = reactive({
  name: '',
  email: '',
  isAdmin: false
})

// no .value — access properties directly
user.name = 'Ahmed'
user.isAdmin = true
```

`ref` vs `reactive` — when to use which?

	REF	REACTIVE
Works with	Anything — primitives, arrays, objects	Objects & arrays only
Access	`.value` in script	Direct property access
Reassignable?	Yes — `ref.value = newObj`	No — breaks reactivity
Destructure?	Yes, with `toRefs`	Loses reactivity

> **My rule:** default to `ref`. It works for everything. Reach for `reactive` only when it clearly reads better.

`computed()` — derived state, cached

```
import { ref, computed } from 'vue'

const cart = ref([
  { name: 'Hat', price: 20, qty: 2 },
  { name: 'Bag', price: 50, qty: 1 }
])

const total = computed(() =>
  cart.value.reduce((sum, i) => sum + i.price * i.qty, 0)
)

// total recomputes only when cart changes — cached otherwise
```

`watch()` — run side effects

```
import { ref, watch } from 'vue'

const query = ref('')

watch(query, (newVal, oldVal) => {
  // runs when query changes — perfect for search, analytics, saving
  console.log('search:', newVal)
})

// watch multiple sources
watch([page, perPage], ([p, pp]) => fetchPage(p, pp))
```

`watchEffect()` — auto-track dependencies

```
import { ref, watchEffect } from 'vue'

const user = ref({ id: 1 })
const page = ref(1)

watchEffect(() => {
  // Vue auto-tracks every reactive thing you READ inside.
  // This re-runs when user.id OR page changes.
  fetchOrders(user.value.id, page.value)
})
```

Easier — but you don't control what it tracks. Use `watch` when you want to be explicit.

⚠️ GOTCHA — REMEMBER THIS

Destructuring kills reactivity

```
const user = reactive({ name: 'Ahmed', age: 25 })

const { name, age } = user        // ❌ now plain values, not reactive

user.name = 'Ghoraba'               // name still says 'Ahmed'
```

You'll hit this with Pinia later. The fix is `toRefs` / `storeToRefs`.

`toRefs()` — the fix

```
import { reactive, toRefs } from 'vue'

const user = reactive({ name: 'Ahmed', age: 25 })

const { name, age } = toRefs(user)   // ✓ now refs — reactivity preserved

user.name = 'Ghoraba'
// name.value === 'Ghoraba' ✓
```

> In Pinia you'll use `storeToRefs(store)` — exact same idea.

DEMO 03

Reactivity.

A counter + derived total. Watch the magic re-run itself.

PART 06 / 15

Components

The heart of Vue.

COMPONENT

Reusable, self-contained UI

A component bundles **HTML + logic + styles** into one reusable unit. Write once, use anywhere.

- **Template** — the HTML the component renders.
- **Script** — the JavaScript that drives it.
- **Style** — CSS scoped to just this component.

ANATOMY

A component, piece by piece

ProductCard.vue

```
<template>
  <div class="card">
    <h3>{{ product.name }}</h3>
    <p>${{ product.price }}</p>
  </div>
</template>

<script setup>
const props = defineProps(['product'])
</script>

<style scoped>
.card { border: 1px solid #eee; padding: 1rem; }
</style>
```

Using a component

ProductList.vue

```vue
<template>
  <ProductCard
    v-for="p in products"
    :key="p.id"
    :product="p"
  />
</template>

<script setup>
import ProductCard from './ProductCard.vue'
import { ref } from 'vue'

const products = ref([ /* … */ ])
</script>
```

An e-commerce page, broken down

```
App.vue
├── HeaderComponent        // nav · logo · cart badge
└── ProductPage
    ├── ImageGallery       // carousel
    ├── ProductDetails     // title · price · desc
    ├── ReviewsSection     // reviews list
    ├── RelatedProducts    // suggestions
    └── FooterComponent    // links · contact
```

DEMO 04

A stupid card.

A component that renders the same thing every time.
We'll make it smart next.

PROPS

Parent → child, data flows down

Props are how a parent component **passes data down** to a child. Declared in the child, sent from the parent's template.

definePropsO

• Child.vue

```
<script setup>
// simple — array of names
const props = defineProps(['product', 'size'])

// better — types, defaults, required, validation
const props = defineProps({
  product: { type: Object, required: true },
  size:    { type: String, default: 'md' },
  qty:     { type: Number, validator: v => v > 0 }
})
</script>
```

⚠️ GOLDEN RULE

Props are read-only

```
const props = defineProps(['product'])

props.product.price = 99          // ❌ never do this

// if the child needs to update something,
// EMIT an event — let the parent do it.
```

Mutating a prop breaks the data-flow model. Vue will warn you in the console — listen to it.

EMITS

Child → parent, events flow up

BuyButton.vue

```
<template>
  <button @click="$emit('buy', product.id)">
    Buy {{ product.name }}
  </button>
</template>

<script setup>
defineProps(['product'])
defineEmits(['buy'])
</script>
```

Parent listens with `@event`

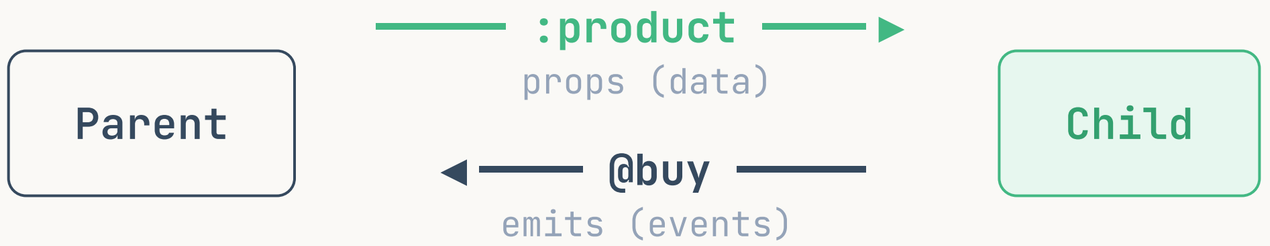
Parent.vue

```vue
<template>
  <BuyButton
    :product="current"
    @buy="handleBuy"              // listen to child's 'buy' event
  />
</template>

<script setup>
function handleBuy(id) {
  console.log('buying', id)             // payload = product.id
}
</script>
```

THE MENTAL MODEL

Props down. Emits up.



The child makes the request. The parent makes the decision. Always.

DEMO 05

Buy button.

Props flowing down. Emits flowing up. Parent owns the cart.

PART 07 / 15

Slots

Components that accept content.

SLOTS

A slot is a hole. The parent fills it.

Props pass *data*. Slots pass *content* — any HTML, any components, anything the parent wants to put there.

Default slot

Card.vue

```vue
<template>
  <div class="card">
    <slot />        // ← hole
  </div>
</template>
```

usage

```vue
<Card>
  <h3>Hello</h3>
  <p>Any markup works here.</p>
</Card>
```

Named slots — multiple holes

Card.vue

```
<div class="card">
  <header><slot name="header" /></header>
  <div class="body"><slot /></div>
  <footer><slot name="footer" /></footer>
</div>
```

usage

```
<Card>
  <template #header>
    <h3>Title</h3>
  </template>

  <p>Body content</p>

  <template #footer>
    <button>Done</button>
  </template>
</Card>
```

Scoped slots — child gives data up to slot

List.vue

```vue
<ul>
  <li v-for="item in items" :key="item.id">
    <slot :item="item" />   // expose item
  </li>
</ul>
```

usage

```vue
<List :items="products">
  <template #default="{ item }">
    <strong>{{ item.name }}</strong>
    — ${{ item.price }}
  </template>
</List>
```

The child owns the data + structure. The parent controls how each item looks.

DEMO 06

Slots.

A Card with header / body / footer slots — used two different ways.

PART 08 / 15

Lifecycle

How components are born and die.

THE PHASES

Every component lives this path

Hooks let you run code at each boundary — fetch data on *mount*, clean up on *unmount*.

Composition API hooks

```
import { onMounted, onUnmounted, onUpdated } from 'vue'

onMounted(() => {
  // DOM is ready — safe to fetch, attach listeners, start timers
  fetchProducts()
})

onUnmounted(() => {
  // clean up — remove listeners, clear timers, cancel requests
  clearInterval(timer)
})

onUpdated(() => {
  // after every re-render. Rarely needed — watch is usually better.
})
```

DEMO 07

On mount.

Fetch a list of products when the component appears.

PART 09 / 15

Vue Router

Many screens. Still one page.

VUE ROUTER

URLs without page reloads

The official routing library. Maps URL paths to components. The browser URL changes, Vue swaps the view — no reload.

You opted into it when scaffolding, so it's already installed.

Defining routes

src/router/index.js

```
import { createRouter, createWebHistory } from 'vue-router'
import Home from '@/views/Home.vue'

const router = createRouter({
  history: createWebHistory(),
  routes: [
    { path: '/',          name: 'home',     component: Home },
    { path: '/products',  name: 'products', component: () => import('@/views/Products.vue') },
    { path: '/about',     name: 'about',    component: () => import('@/views/About.vue') }
  ]
})

export default router
```

`<RouterLink>` — never plain ~~`<a>`~~

```vue
// ✓ by path
<RouterLink to="/products">Shop</RouterLink>

// ✓ by name with params
<RouterLink :to="{ name: 'product', params: { id: 42 }}">
  Details
</RouterLink>

// ❌ plain <a> causes a full page reload
<a href="/products">Shop</a>   // kills all your state
```

External links and `target="_blank"` are fine. Internal nav → always RouterLink.

`<RouterView>` — where routes render

App.vue

```
<template>
  <Navbar />
  <main>
    <RouterView />      // ← current route's component goes here
  </main>
  <Footer />
</template>
```

Programmatic navigation

```
import { useRouter } from 'vue-router'

const router = useRouter()

async function login() {
  await authenticate()
  router.push('/dashboard')              // by path
  router.push({ name: 'dashboard' })     // by name
  router.replace('/home')                  // no history entry
  router.back()                            // go back
}
```

Route params vs query

```
// route definition
{ path: '/products/:id', component: ProductDetails }

// URL:  /products/42?ref=newsletter
import { useRoute } from 'vue-router'
const route = useRoute()

route.params.id       // "42"  — always a STRING
route.query.ref       // "newsletter"
```

params

Part of the path. For IDs, slugs, required data.

query

After `?`. For filters, pagination, optional data.

Nested routes — shared layouts

```
{
  path: '/dashboard',
  component: DashboardLayout,      // has its own <RouterView />
  children: [
    { path: '',        component: DashHome },        // /dashboard
    { path: 'orders',  component: DashOrders },      // /dashboard/orders
    { path: 'settings',component: DashSettings }
  ]
}
```

Each child renders inside the parent's RouterView. Sidebar stays, only the inner pane swaps.

Navigation guards — for auth

```
router.beforeEach((to, from) => {
  const auth = useAuthStore()
  if (to.meta.requiresAuth && !auth.user) {
    return { name: 'login', query: { redirect: to.fullPath } }
  }
})

// mark routes:
{ path: '/admin', component: Admin, meta: { requiresAuth: true } }
```

Lazy-load routes — smaller bundles

```
// ❌ eager — every page in the initial bundle
import Admin from '@/views/Admin.vue'
{ path: '/admin', component: Admin }

// ✓ lazy — loaded only when the user visits /admin
{ path: '/admin', component: () => import('@/views/Admin.vue') }
```

Vite code-splits the dynamic import automatically. Free performance.

DEMO 08

Router.

Three routes, a nested dashboard layout, a guarded admin page, a lazy route.

PART 10 / 15

HTTP & APIs

Apps without data are demos.

`fetch` vs `axios`

fetch

- Built into the browser. Zero install.
- Two-step JSON: `await res.json()`.
- Doesn't throw on 4xx/5xx — you check `res.ok`.

axios

- `npm install axios`.
- Auto-JSON, cleaner API.
- Interceptors for auth headers, logging, retries.

Pick one. Be consistent.

REFRESHER

async / await + try / catch

```
async function loadProducts() {
  try {
    const res = await fetch('/api/products')
    if (!res.ok) throw new Error('Request failed')
    return await res.json()
  } catch (err) {
    console.error(err)
    throw err
  }
}
```

Always wrap `await` in try/catch. Promises don't throw like normal code — an unhandled rejection is a silent bug.

Every API call has three states

Loading

Show a spinner / skeleton. An empty screen looks broken.

Success

Render the data. The happy path.

Error

Show a real message. Offer a retry.

The `useFetch` pattern

composables/useFetch.js

```
import { ref } from 'vue'

export function useFetch(url) {
  const data    = ref(null)
  const error   = ref(null)
  const loading = ref(true)

  async function run() {
    loading.value = true; error.value = null
    try {
      const res = await fetch(url)
      data.value = await res.json()
    } catch (e) { error.value = e }
    finally { loading.value = false }
  }

  run()
  return { data, error, loading, refetch: run }
}
```

ARCHITECTURE

Where do API calls live?

- **Small apps** — inline in the component's `onMounted`.
- **Medium apps** — in **composables**. One per resource.
- **Big apps** — in **Pinia store actions**. State + calls colocated.

> **Never** call the API from the template. Templates are declarative. Side effects belong in script.

DEMO 09

useFetch.

Reusable fetch. Loading, error, success. Two components, one composable.

PART 11 / 15

Composables

Functions that work *with* Vue.

THE PROBLEM

Prop drilling is misery

```
App.vue
└── HomeView                    // owns cart
    └── ProductPage              // ← needs cart count
        └── ProductCard
            └── BuyButton        // ↑ emit up 4 levels
```

Passing data through five layers that don't care about it. Fragile. Noisy. Makes you hate your life.

THE FIX (PART 1)

A composable is a function

- A plain JavaScript function …
- … that uses Vue's reactivity inside (`ref`, `computed`, `watch`).
- Encapsulates **reusable logic** — not UI.
- Named with `use` prefix by convention.
- Lives in its own `.js` or `.ts` file — not `.vue`.

DEMO 10

useLocalStorage.

30 lines of code. Reactive ref synced to browser storage.
Used everywhere.

PART 12 / 15

Pinia

Shared state. Any component. No drilling.

Composables share logic. Stores share state.

Every component that calls `useCounter()` gets its *own* counter. A **store** is one shared source of truth the whole app reads and writes.

Think of it as `ref` *that lives outside any component.*

Setup store (our pick)

stores/cart.js

```
import { defineStore } from 'pinia'
import { ref, computed } from 'vue'

export const useCartStore = defineStore('cart', () => {
  // state
  const items = ref([])

  // getters
  const total = computed(() =>
    items.value.reduce((s, i) => s + i.price * i.qty, 0)
  )

  // actions
  function add(product) { items.value.push({ ...product, qty: 1 }) }
  function clear() { items.value = [] }

  return { items, total, add, clear }
})
```

⚠️ THE PINIA GOTCHA

Using a store — mind the destructure

```
import { storeToRefs } from 'pinia'
import { useCartStore } from '@/stores/cart'

const cart = useCartStore()

// ❌ loses reactivity on state / getters
const { items, total } = cart

// ✓ state and getters → storeToRefs
const { items, total } = storeToRefs(cart)

// ✓ actions → destructure normally
const { add, clear } = cart
```

Pinia vs props

	PROPS	PINIA
`Where data lives`	Parent component	Store (outside components)
`Access`	Pass through every layer	Call the store directly
`Updates everywhere?`	Only if wired correctly	Yes — automatically
`Best for`	Local component data	Shared app-wide state

Pinia + localStorage — the real pattern

Pinia alone

Reactive. Fast. **Resets on refresh.**

localStorage alone

Persists. **Not reactive.**

Together

Reactive **AND** persistent. This is how real e-commerce carts work.

DEMO 11

Cart that survives reload.

Pinia store + useLocalStorage. Refresh. Items still there.

Because we can.

PART 13 / 15

Forms

Every app has them.

Hand-rolled validation

```vue
<script setup>
import { reactive, computed } from 'vue'

const form = reactive({ email: '', password: '' })

const isValid = computed(() =>
  form.email.includes('@') && form.password.length >= 8
)
</script>

<template>
  <input v-model="form.email" />
  <input v-model="form.password" type="password" />
  <button :disabled="!isValid">Sign in</button>
</template>
```

IN REAL PROJECTS

VeeValidate + Zod

VeeValidate

Handles form state, submit flow, error display. Vue-native.

Zod

Defines the schema. "Email must be string, at least 8 chars, etc."

Pair them. Zod describes the shape, VeeValidate wires it into your form. But — **understand the hand-rolled version first.**

PART 14 / 15

TypeScript · Env · Deploy

Things to know exist.

It just works

```vue
<script setup lang="ts">
interface Product {
  id: number
  name: string
  price: number
}

const props = defineProps<{ product: Product }>()
const emit = defineEmits<{ buy: [id: number] }>()

const count = ref<number>(0)
</script>
```

`defineProps` and `defineEmits` get full type inference. Use TS after today — it's worth it.

Environment variables

• .env

```
VITE_API_URL=https://api.myshop.com
VITE_STRIPE_KEY=pk_test_xxx
```

• usage

```
const url = fetch(
  `${import.meta.env.VITE_API_URL}/products`
)
```

Must start with `VITE_`. **Never put secrets in the frontend** — env vars are bundled into your public JS.

Build & deploy

```
npm run build       // → dist/ folder with static HTML + JS + CSS
npm run preview     // test the built version locally
```

Netlify

Drag dist/ in. Live in 20 seconds.

Vercel

Connect your repo. Auto-deploy on push.

Cloudflare Pages

Free tier is wild. Global edge.

PART 15 / 15

The Map

You have the foundation. Here's what's next.

WHERE TO GO FROM HERE

Level up

- **Vitest + Vue Test Utils** — test your components.
- **Nuxt** — SSR, SEO, file-based routing.
- **TypeScript** — seriously, learn it.
- **VueUse** — a massive library of pre-built composables.
- **PrimeVue** / **Vuetify** — component libraries.
- **Vue Query** / **TanStack Query** — server state done right.
- **Pinia plugins** — persistence, devtools, hydration.
- **Performance** — `<Suspense>`, `v-memo`, `Teleport`.

PLEASE. I MEAN IT.

Read the docs.

vuejs.org · Reading docs fluently is a skill on its own. Practice it.

Thank you.

Ahmed Ghoraba · SWE / ITI TA

Questions? Now's the time.